

cMCP — Study Guide

The Model Context Protocol, and how a from-scratch C11 library implements it.
A document to *learn* the protocol and to *read* this codebase.

Generated 2026-06-09 • cMCP v0.9.0 • protocol 2025-11-25 • pure C11

How to read this. Part I teaches the Model Context Protocol from first principles — no C required, MCP as a specification. Part II walks the cMCP source tree layer by layer and shows *where* each protocol concept is implemented. Part III is a guided reading order plus hands-on exercises. If you already know MCP, skim Part I for cMCP’s vocabulary and jump to Part II. The companion documents are `docs/architecture.md` (the “why” of the partitioning, in prose) and `docs/cmcp-engineering-report.tex` (the *measured* conformance / performance / hardening claims). This guide is the *didactic* layer over both.

Contents

I	The Model Context Protocol	1
1	What MCP is, and the problem it solves	1
2	The JSON-RPC 2.0 foundation	1
3	Lifecycle: handshake and capability negotiation	2
4	The three server primitives	3
5	The reverse direction: server-initiated requests	3
6	Transports	4
II	Inside cMCP	4
7	Build, link targets, and the six-layer pipeline	4
8	Layer by layer	5
8.1	json.c — the value tree (§6, layer 1)	5
8.2	rpc.c — JSON-RPC framing (layer 2)	5
8.3	schema.c — the JSON Schema subset (layer 3)	5
8.4	transport_*.c — the vtable (layer 4)	5
8.5	server.c + worker.c — the server half (layer 5)	6
8.6	client.c + session.c — the client half (layer 5)	6
9	The threading model	6
10	Memory ownership	7
11	Security and hardening	7
12	How correctness is proved	8

III A guided study path	8
13 Reading order	8
14 Hands-on exercises	8
15 Glossary	9

Part I. The Model Context Protocol

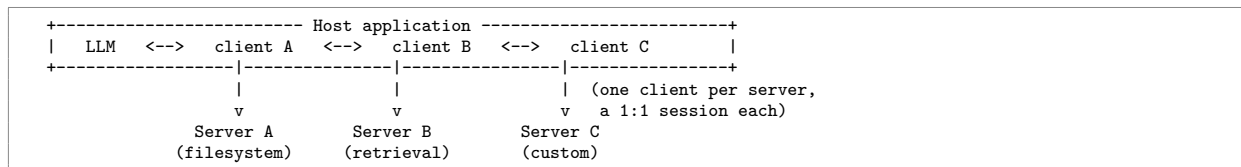
1 What MCP is, and the problem it solves

A large language model is a function from text to text. To do anything in the world — read a file, search a corpus, call an API — it needs an external program to act on its behalf and feed results back into its context window. Before MCP, every host application (an IDE, a chat client, an agent) invented its own bespoke plugin protocol, and every tool author re-implemented against each host. That is an $M \times N$ integration problem.

The **Model Context Protocol** (introduced by Anthropic as an open standard) collapses it to $M + N$: tools speak one protocol, hosts speak one protocol, and any host can drive any tool. The common analogy is “a USB-C port for AI applications” — a single physical shape that many devices plug into.

Three roles. An MCP deployment has exactly three kinds of participant:

- **Host** — the application the user interacts with, and the thing that owns the LLM (e.g. Claude Desktop, an IDE, or cMCP’s target consumer, the *butlerbot* agent). The host decides which servers to connect to and whether to honour a server’s requests.
- **Client** — the connector *inside* the host that maintains a *one-to-one* session with a single server. A host with three servers runs three clients. In cMCP this is `libcmcp_client.a`.
- **Server** — a separate program that exposes capabilities (tools, resources, prompts) over the protocol. In cMCP this is `libcmcp_server.a`; the reference servers are `filesystem-mcp` and `crag-mcp`.



The key mental model: the **client/server link is symmetric in machinery but asymmetric in role**. Both sides frame the same JSON-RPC, negotiate capabilities, and can send unsolicited messages — but the server *offers* primitives and the client *drives* them. cMCP exploits this directly: ~60% of the code (framing, schema, transport, capability structs) is shared in `libcmcp_core.a`, and only the asymmetric halves live in the server and client libraries.

2 The JSON-RPC 2.0 foundation

Every MCP message is a JSON-RPC 2.0 object. There are exactly three shapes, and understanding them is most of understanding the wire.

1. **Request** — carries "id", "method", optional "params". Demands a response. *Example:* `{"jsonrpc":"2.0","id":7,"method"}`
2. **Response** — carries the same "id" and *either* "result" (success) *or* "error" (never both).
3. **Notification** — a request with *no* "id". Fire and forget; no response is ever sent. *Example:* `notifications/initialized, notifications/cancelled`.

The id space. Ids match responses to requests. Each session has a single monotonic id space. The side that *sends* a request owns the id and correlates the inbound response by it. Because both sides may send requests

(the server can ask the host for an LLM completion, see §5), there are *two* id spaces per session — one per direction — and they are independent.

Error codes. JSON-RPC defines a standard set, all reused by MCP:

Code	Name	Meaning
-32700	Parse error	Invalid JSON received
-32600	Invalid request	Not a valid Request object (e.g. a batch, which MCP forbids)
-32601	Method not found	Unknown method, or a default-deny decline
-32602	Invalid params	Bad/missing params (e.g. malformed <code>protocolVersion</code>)
-32603	Internal error	Server-side failure (e.g. structured output fails its schema)

No batching. JSON-RPC 2.0 permits arrays of messages; MCP *removed* batch support (since spec 2025-06-18). A conformant implementation parses a batch only so it can reject it cleanly with -32600.

The dual error-channel subtlety (load-bearing). There are *two* distinct ways a `tools/call` can “fail,” and a correct host must check both:

- A **protocol-level** failure (unknown tool, malformed params) is a JSON-RPC **error** object with code `-32602/-32601`.
- A **tool-level** failure (the tool ran but its arguments were invalid, or it errored semantically) is a *successful* JSON-RPC **result** whose body is `{isError:true, content:[{text:"..."}]}`.

This split is deliberate in the spec: “the tool could not be invoked” is a protocol error; “the tool ran and reports a problem” is data the model should see. cMCP matches the TypeScript reference SDK here and pins it by test (§13).

3 Lifecycle: handshake and capability negotiation

An MCP session has a strict opening sequence. Nothing operational happens until it completes.

```

client                                                                    server
-----                                                                    -----
initialize (protocolVersion, clientInfo, capabilities) ----->
    <----- result (protocolVersion, serverInfo, capabilities)
notifications/initialized ----->
===== session is now "ready"; normal traffic =====

```

Step 1 — initialize. The client sends its protocol version, an `clientInfo {name, version}`, and a *capabilities* object declaring what *it* can do (sampling? roots? elicitation?). The server replies with its own version, `serverInfo`, and *its* capabilities (tools? resources? prompts? logging?).

Step 2 — notifications/initialized. The client confirms it processed the result. Only now may either side issue operate-class requests. A server that receives, say, `tools/call` before `initialized` answers `-32600`.

Version tolerance (a common mistake). The protocol version is a date string (cMCP pins 2025-11-25). If the peer advertises a *different* version, this is **not** a handshake failure: the spec says respond with success, advertise your own version, and let the *other* side decide whether to disconnect. Only a *missing or malformed* protocolVersion is a hard -32602. cMCP captures the peer's version (cmcp_client_server_protocol) and proceeds.

Capability negotiation is the contract. A capability not advertised must not be used. If a server never declared `resources.subscribe`, the client must not call `resources/subscribe`; if a client never declared `sampling`, the server must not ask it for a completion. This is the mechanism that keeps both ends from sending messages the other isn't prepared to handle. Sub-capabilities (`listChanged`, `subscribe`, `logging`) are opt-in flags nested under the top-level capability.

4 The three server primitives

A server exposes three kinds of thing. Each has a `*/list` method to enumerate it and a method to act on one. Knowing who controls each is the clarifying lens (the spec frames this as *who is in the driver's seat*):

Primitive	Control model	List	Act
Tools	model-controlled	<code>tools/list</code>	<code>tools/call</code>
Resources	application-controlled	<code>resources/list</code>	<code>resources/read</code>
Prompts	user-controlled	<code>prompts/list</code>	<code>prompts/get</code>

Tools are functions the *model* chooses to invoke. Each carries a `name`, a human `description`, and an `inputSchema` (JSON Schema) describing its arguments. The model reads the schema, fills arguments, and the host issues `tools/call`. Results are a `content` array of typed items (`text`, `image`, `resource_link`, ...), optionally a typed `structuredContent` validated against an `outputSchema`.

Resources are data the *application* pulls into context — a file, a database row, a diagnostics blob. Each has a `uri` (e.g. `file:///path`, `crag://stats`). The host reads them with `resources/read`. A client may `resources/subscribe` to a URI and receive `notifications/resources/updated` when it changes (only if the server declared `resources.subscribe`).

Prompts are reusable, parameterised message templates the *user* invokes (think slash-commands). `prompts/list` enumerates them with a flat argument descriptor list `[{name, description?, required?}]` — note this is *not* full JSON Schema — and `prompts/get` expands one with arguments into a message list.

list_changed. If a server’s primitive set is dynamic, it declares `listChanged` and emits `notifications/tools/list_changed` (etc.) so the client knows to re-list.

5 The reverse direction: server-initiated requests

MCP is bidirectional. Beyond responding to the client, a server can originate requests *back to the host* — this is what makes MCP an agent protocol rather than an RPC catalogue. There are three such directions, all gated by a *client* capability:

- **Sampling** (`sampling/createMessage`) — the server asks the host’s LLM to produce a completion. Useful for a tool that wants to summarise or re-reason over raw output before returning it. Gated by the client’s `sampling` capability; default-decline (`-32601`) if the host registered no handler.
- **Roots** (`roots/list`) — the server asks the host which filesystem paths / URIs are in scope. A filesystem-shaped server consults this before touching anything. The host declares `roots` and may emit `notifications/roots/list_changed`.
- **Elicitation** (`elicitation/create`) — mid-tool-call, the server asks the user for structured input: a confirmation, a missing argument, a credential. The request carries a `requestedSchema`; the host returns `{action:"accept"|"decline"|"cancel", content}`.

Two more cross-cutting mechanisms ride alongside:

- **Progress** — a long call attaches a `progressToken` (in `params._meta`); the server emits `notifications/progress` carrying that token, and the host correlates it to the originating call.
- **Cancellation** — either side may send `notifications/cancelled {requestId, reason?}` to abandon an in-flight request. It is *cooperative*: the receiver should stop work, but a response that crosses the cancel on the wire is simply dropped.
- **Logging** — with the `logging` capability, a server ships structured `notifications/message {level, logger?, data}` events, and the host dials the floor with `logging/setLevel`. Levels are the eight RFC 5424 syslog levels.

The trust model. Every reverse request is default-deny. The host opts in *per server* by registering a handler — “did the host bother to register a handler for this server?” is the authorisation gate. There is no global allow-list; trust is decided one server at a time.

6 Transports

MCP separates message *semantics* from message *transport*. The spec defines two standard transports; cMCP implements both.

- **stdio** — the server is a subprocess; the host writes JSON-RPC to its stdin and reads from its stdout, one message per line (newline-delimited). *stdout is sacred* — any stray write corrupts the wire, so all logging

goes to stderr. This is the primary transport for local, single-tenant tools and cMCP’s design centre.

- **Streamable HTTP** — a single /mcp endpoint. The client POSTs a request and gets the response in the body; it may also open a GET with `Accept: text/event-stream` to receive server-initiated messages over Server-Sent Events (SSE). A session id (`Mcp-Session-Id` header) is minted on the first `initialize` and threaded through subsequent calls. SSE events carry monotonic ids so a dropped stream can resume via `Last-Event-Id` (spec SEP-1699).

Why stdio is the default for agents. A local tool launched as a subprocess needs no ports, no auth, no TLS — the OS process boundary *is* the security boundary, and stdin/stdout are already private to the parent. HTTP matters for remote/shared servers but brings a network attack surface (§12).

Part II. Inside cMCP

7 Build, link targets, and the six-layer pipeline

cMCP is one source tree compiled into three static libraries plus reference binaries. The partitioning mirrors the protocol’s shared-machinery / asymmetric-role split from §1.

libcmcp_core.a	json + rpc + schema + types + transports	(shared ~60%)
libcmcp_server.a	+ server.c + worker.c	(depends on core) (server half)
libcmcp_client.a	+ client.c + session.c	(depends on core) (client half)

Link order matters: consumers (server/client) before provider (core), so the linker resolves core symbols on the second pass. The Makefile handles it. The core libraries are dependency-free; only the HTTP *client* links libcurl.

Build commands to know:

Command	Effect
<code>make</code>	libs + <code>cmcp-inspect</code> + <code>filesystem-mcp</code> + <code>cmcp-tee</code> + examples
<code>make test</code>	build and run every <code>tests/test_*.c</code> (offline, hermetic)
<code>make valgrind</code>	same suite under Valgrind (leak-free)
<code>make test-asan</code>	rebuild under ASan/UBSan, run suite
<code>make test-tsan</code>	rebuild under ThreadSanitizer, run suite
<code>make crag-mcp</code>	build the cRAG reference server (needs sibling ../cRAG)
<code>make conformance</code>	cross-check vs the TS reference SDK (needs Node + network)

The request pipeline is six layers, each in its own module, each talking only to the layer directly below:

+-----+ tools (handlers) +-----+ server.c - registry, dispatch client.c - async client + worker.c pool + session.c N-server mux +-----+ schema.c - JSON Schema subset (validates tools/call.arguments) +-----+ rpc.c - JSON-RPC 2.0 framing, in-flight ID table +-----+ transport_stdio.c / transport_http.c / transport_http_client.c +-----+ json.c - typed value tree, parser, emitter +-----+	<- user code
---	--------------

The discipline is strict: the transport layer never inspects message content; the RPC layer never reads raw bytes; the schema layer never speaks JSON-RPC. That clean separation is what lets the same RPC core serve both halves and lets HTTP plug in on the same vtable as stdio.

8 Layer by layer

8.1 json.c — the value tree (§6, layer 1)

A hand-written JSON parser/emitter over a discriminated-union value tree (`cmcp_json_t`). No third-party JSON library. Lifted from cRAG's `util.c` and extended with object/array constructors, escape-aware string emission (UTF-8 surrogate pairs, control bytes), and *stable key ordering on emit* — keys come out in insertion order. That last is not a JSON requirement but it makes wire frames byte-stable, which is what lets tests and the replay gate diff frames literally.

Hardening lives here too: `CMCP_JSON_MAX_DEPTH` (default 64) bounds recursion against stack exhaustion, and `CMCP_JSON_MAX_ELEMENTS` (default 65536) caps array/object size against memory bombs.

8.2 rpc.c — JSON-RPC framing (layer 2)

A discriminated-union message type (`cmcp_rpc_message_t`) covering request / response / notification, plus emit and parse. It parses batches only to reject them with `-32600` (§2). The *in-flight ID table* (`cmcp_rpc_pending_t`) is the single monotonic id space: the client reserves an id before sending, the reader looks the id up on the inbound response and routes the parsed message to whoever waits. `CMCP_RPC_MAX_INFLIGHT` (default 1024) caps outstanding ids.

8.3 schema.c — the JSON Schema subset (layer 3)

The server validates inbound `tools/call.arguments` against the tool's registered `inputSchema` *before* the handler runs — so handlers only ever see well-formed input. The same validator runs *outbound* when a handler attaches `structuredContent` against an `outputSchema` (mismatch $\rightarrow$ `-32603`).

Supported keywords: `type` (incl. array-of-types), `properties`, `required`, `additionalProperties>false`, `enum`, `minLength/maxLength` (Unicode code points), `minimum/maximum`, `items`. *Deliberately unsupported* in v0.1: `$ref`, `oneOf/anyOf/allOf`, `format`, `pattern`. This is a documented subset (`docs/schema-conformance.md`), not a silent gap — those keywords would multiply validator complexity for cases real MCP tools don't need.

Recall the dual channel from §2: a `tools/call argument` failure surfaces as a tool-level `{isError:true}` result, while protocol-level rejections use the `-32602` error channel.

8.4 transport_*.c — the vtable (layer 4)

All three transports implement one vtable, `cmcp_transport_t`: `read_fn`, `write_fn`, `close_fn`, and an optional `wake_fn`.

stdio (`transport_stdio.c`) — newline-delimited JSON over a (`read_fd`, `write_fd`) pair. `getline()` for reads (tolerates blank lines, grows with the message); `fwrite + '\n' + fflush` for writes under a per-transport mutex so a reply and a notification thread never interleave a frame. `CMCP_STDIO_MAX_FRAME` (default 16 MiB) caps a frame so a peer streaming endlessly without a newline can't OOM the process — this protects the *client* too, since it reads a forked server's stdout through the same path.

HTTP server (`transport_http.c`) — a hand-rolled HTTP/1.1 server on `socket()/accept()`. One /mcp endpoint: `POST` $\rightarrow$ request/response, `GET` with SSE $\rightarrow$ server-push channel. It bridges the synchronous `read $\rightarrow$ dispatch $\rightarrow$ write` server loop onto HTTP with a single-slot mailbox (there is only ever one logical session per transport). This is the one place the transport layer peeks the JSON-RPC body — to tell a notification (202 Accepted, no body) from a request (200 + body), since HTTP forces that decision.

HTTP client (`transport_http_client.c`) — the mirror, built on `libcurl`. Each `write_fn` is a one-shot `POST` with its own easy handle (so concurrent host threads don't serialise), and a separate SSE thread long-polls `GET /mcp` for server-pushed frames, pushing both onto one read queue that `client.c` demultiplexes. The `wake_fn` slot exists because this transport's `read_fn` parks on a `condvar`, not a `syscall` (§10).

The HTTP parser itself lives in `http_parser.c` / `cmcp_http_parser.h`, extracted as its own translation unit so a libFuzzer harness can drive it without sockets.

8.5 server.c + worker.c — the server half (layer 5)

`server.c` holds the tool / resource / prompt registries, the dispatch table, and the run loop. Built-in methods: the handshake, `ping` (answered `{}` *before* the readiness gate so it works pre-handshake), all three `*/list` + `act` methods, `resources/subscribe/unsubscribe`, and `logging/setLevel`. Unknown methods $\rightarrow$ `-32601`; operate-class methods *before* `initialize` $\rightarrow$ `-32600`.

Capabilities `tools/resources/prompts` are *auto-advertised* whenever ≥ 1 of that kind is registered; sub-

capabilities (subscribe, listChanged, logging) stay opt-in via `cmcp_server_set_capabilities`. The minimal server is small:

```
cmcp_server_t *s = cmcp_server_new("echo", "1.0");
cmcp_server_add_tool(s, "echo", "Echo text back",
    "{\"type\":\"object\",\"properties\":{\"text\":{\"type\":\"string\"}},\"required\":[\"text\"]}",
    echo_handler, NULL);
cmcp_server_run_stdio(s); /* blocks: read -> dispatch -> write */
```

`worker.c` is the handler thread pool. The three handler-*invoking* methods (`tools/call`, `resources/read`, `prompts/get`) run on `CMCP_WORKERS` threads (default 4) so a slow handler can't stall the read loop; everything else (handshake, lists, notifications) runs inline on the loop thread, so the lifecycle FSM needs no lock. See §10 for the cancellation and timeout machinery.

8.6 client.c + session.c — the client half (layer 5)

`client.c` is an async client with *one reader thread* per `cmcp_client_t`. Host threads call `cmcp_client_call_async` (returns an id) and later `cmcp_client_wait(id)`; `cmcp_client_request` is just those two fused. The reader demultiplexes inbound frames by kind: responses go to the matching pending entry; `notifications/progress` go to a per-call subscription if its token matches, else to the generic handler; reverse requests (`sampling/elicitation/roots/ping`) go to the registered host handler or default-decline.

`session.c` aggregates N already-handshaken clients under host-supplied names so a multi-server host sees one flat surface. Tools are namespaced `<server>:<tool>`; `resources/read` and `prompts/get` take an explicit `(server, uri|name)` pair (URIs already contain colons, so folding the server in would be ambiguous). List calls fan out async, fan in sequentially, and follow each server's `nextCursor` pagination automatically.

9 The threading model

This is the subtlest part of the codebase and the most worth studying.

Server side. The run loop owns the transport on one thread: read, parse, dispatch. Dispatch forks two ways — cheap/lifecycle methods run *inline* (so the FSM is lock-free), and the three handler-invoking methods go to a bounded worker pool. The pool's submit *blocks* when full, so backpressure propagates to the loop instead of growing memory.

Cooperative cancellation (a contract, not a kill). There is deliberately no `pthread_cancel` — async-killing a thread that may hold the writer mutex or a `malloc` lock is unsafe in C. Instead an in-flight table maps request id → ctx; `notifications/cancelled` or a 200ms watchdog (past `CMCP_HANDLER_TIMEOUT_MS`, default 30s) only *flag* a context. The handler contract is therefore load-bearing: **a handler MUST poll `cmcp_handler_cancelled()` in any loop or before any blocking step**. A handler that blocks forever burns its worker slot permanently; `CMCP_WORKERS` such handlers deadlock the server.

Client side. One reader, many waiters. Each call gets a completion record with its own mutex+condvar (localised wakeup, no thundering herd). Multiple async calls may be in flight and `wait` may be called in any order.

caller thread(s)	reader thread
	for (;;) {
call_async:	transport_read()
register pending id	rpc_parse()
alloc + link completion record	switch (kind):
send_message	RESPONSE -> deliver by id, broadcast cv
return id	NOTIFICATION -> progress sub? else notif_fn
wait(id):	REQUEST -> ping/sampling/elic/roots
cond_wait until done	else -32601
move response out, free	}

Two shutdown subtleties worth internalising. (1) On Linux, closing an fd does *not* interrupt a `read()` already blocked on it. So the stdio reader is woken by a signal (`SIGUSR2`, non-restarting handler) that makes the syscall return `EINTR`; a `pthread_tryjoin_np` retry loop re-sends the signal to close a race window. (2) A transport whose `read_fn` parks on a *userspace* primitive (the HTTP client's condvar) can't be woken by a signal — that's why the vtable has the optional `wake_fn`, which stdio leaves NULL.

Callbacks run on the reader thread, serialised. So a slow host callback (sampling doing an LLM round-trip) stalls inbound frames until it returns. From inside a callback the non-blocking calls (`call_async`, `notify`, `cancel`) are safe, but the blocking pair (`request`, `wait`) self-deadlocks — the thread that would complete them is the one running the callback.

10 Memory ownership

Manual `malloc/free` throughout; the rule is a single convention.

Object	Who frees
<code>cmcp_json_t*</code> passed into <code>*_set/*_request</code> /handlers	callee (library)
<code>cmcp_json_t*</code> returned from <code>*_get</code>	nobody — borrowed, lives with parent
<code>cmcp_rpc_message_t</code>	holder; <code>cmcp_rpc_message_clear</code> before scope exit
Tool-handler <code>*out_content</code>	library, after emitting the response
Tool-handler <code>arguments</code>	caller (library) — borrowed, handler must NOT free
Owned transport (<code>connect_stdio</code>)	client, on <code>client_free</code>
Spawned child process	client, reaped via <code>SIGTERM</code> + <code>waitpid</code>
Clients added to a session	session, on <code>session_free</code>

The convention: any function whose docs say “consumed” or “takes ownership” is a transfer point. Everything else is borrowed. Negative return values are `cmcp_err_t` codes (`include/cmcp.h`); the library never `exit()`s — a handler must survive a single bad call without taking the process down.

11 Security and hardening

cMCP’s trust model is precise: the *host* is trusted (cMCP runs in its address space), but the *peer* on the other end of a transport may be hostile or buggy, and its input is untrusted. A STRIDE threat catalogue maps to concrete, individually-tested controls — each reachable from a snapshot-once environment knob so an operator can tune or disable it without recompiling.

Threat	Control	Knob
SSRF via discovery	egress guard on the <i>resolved</i> peer; rejects metadata/link-local/RFC1918/CGNAT/ULA; DNS-rebinding-safe	<code>CMCP_HTTP_ALLOW_PRIVATE</code>
DNS rebinding	Origin allow-list on the HTTP server	<code>CMCP_HTTP_ALLOWED_ORIGINS</code>
Insecure deserialization	schema validation <i>before</i> dispatch; parse depth + element caps	<code>CMCP_JSON_MAX_DEPTH/_ELEMENTS</code>
Memory bomb / OOM	stdio frame cap, HTTP body cap, handler <code>RLIMIT_AS</code>	<code>CMCP_STDIO_MAX_FRAME</code>
Slowloris / spike	per-recv idle timeout, cumulative deadline, accept-rate token bucket	<code>CMCP_HTTP_ACCEPT_RATE</code>
Credential leakage	log redactor over credential-shaped keys	<code>CMCP_LOG_REDACT</code>

The controls are layered front-to-back: a request crosses the **Origin** allow-list and SSRF guard, then frame/body/depth/element caps, then schema validation, then rate-limit and idle/deadline timeouts, before a handler runs under `RLIMIT_AS`; anything it logs passes the credential redactor on the way out.

Deliberate non-goals (scope, not omission). cMCP links *no* TLS library — a protocol library is the wrong layer to own cert rotation; put `nginx/caddy/Envoy` in front (`docs/deployment-tls.md`). Application-layer auth (OAuth 2.1, API keys, mTLS) is the terminator’s job; cMCP trusts the already-authenticated request. Out-of-process handler sandboxing is a separate deferred tier — today’s threat model is single-author tools, not untrusted third-party code. The network surface is *defensive depth* for a deployment that is stdio-primary and single-tenant.

12 How correctness is proved

For a protocol library the central question is not “does it run” but “does it emit *exactly* the bytes the spec requires, and interoperate with an independent implementation.” cMCP answers with cross-implementation checks in both wire roles, not self-consistency alone.

- **cMCP client vs. TS reference server** and **TS client vs. cMCP server** (`make conformance`) — both directions against the official TypeScript SDK.
- **Replay gate** (`make replay`) — real wire transcripts, captured through the `cmcp-tee` proxy, replayed and diffed so a framing change can't regress silently.
- **Spec-drift watch** (`make check-spec-drift`, weekly CI) — compares the pinned `CMCP_PROTOCOL_VERSION` against the upstream spec.
- **Sanitisers + fuzzers** — the suite runs clean under Valgrind, ASan/UBSan, and ThreadSanitizer; four libFuzzer harnesses drive the JSON, JSON-RPC, schema, and HTTP-head parsers; a soak driver proves no drift over long stdio/HTTP runs.

The tests are not mocked: `test_stdio_roundtrip` actually `fork()`s a child server and runs a real handshake; `test_client_server` wires real `cmcp_server_run` instances onto pipe pairs to exercise the async client and multi-server session end-to-end. The suites map onto the classic Unit → Integration → Evaluation pyramid, with the security axis cutting across all layers.

Part III. A guided study path

13 Reading order

Read the code in the order data flows, bottom-up, with a header before each implementation:

1. `include/cmcp.h` — version, protocol, the `cmcp_err_t` codes. Five minutes; it frames everything.
2. `include/cmcp_json.h` then `src/json.c` — the value tree is the vocabulary every other layer speaks.
3. `include/cmcp_types.h` + `src/rpc.c` — the three message shapes and the id table.
4. `include/cmcp_schema.h` + `src/schema.c` — alongside `docs/schema-conformance.md` for the subset boundary.
5. `include/cmcp_transport.h` + `src/transport_stdio.c` — the vtable, on its simplest backend, before the HTTP backends.
6. `include/cmcp_server.h` + `src/server.c` (then `src/worker.c`) — the registry, dispatch, and pool.
7. `include/cmcp_client.h` + `src/client.c` (then `include/cmcp_session.h` + `src/session.c`) — the reader-thread demux and multi-server aggregator.
8. `examples/echo-server.c` + `examples/minimal-client.c` — the smallest end-to-end pair; trace a `tools/call` through every layer above.
9. Only then the HTTP backends (`transport_http.c`, `transport_http_client.c`) — the most machinery, least protocol.

Keep `docs/architecture.md` open as the prose “why” companion to the headers’ “what.”

14 Hands-on exercises

1. **Build and self-test.** `make && make test`. Then run one binary directly, e.g. `./tests/test_schema`, and open its source to see the assertion style.
2. **Watch a real handshake.** Use `cmcp-inspect` to spawn `filesystem-mcp`, run the handshake, and list its tools/resources. Read the `initialize` exchange it prints.
3. **Capture the wire.** Put `cmcp-tee` between `cmcp-inspect` and a server; inspect the JSONL log to see every frame in both directions — this is exactly what the replay gate diffs.
4. **Write a tool.** Add a second tool to `examples/echo-server.c` with a non-trivial `inputSchema`, rebuild, and call it with deliberately bad arguments. Confirm the failure arrives as a tool-level `{isError:true}` result, *not* a JSON-RPC error (§2, §9).
5. **Break a control.** Set `CMCP_JSON_MAX_DEPTH=4` and feed a deeply-nested object; watch the parser fail closed with `-32700`. Then explore the other knobs in the §12 table.

6. **Prove thread-safety.** Run `make test-tsan` and read `tests/test_client_server.c` to see how concurrent async calls are exercised.

Build this document: `pdflatex docs/cmcp-study-guide.tex` (run twice so the table of contents resolves). It needs no external figures.

15 Glossary

Host

The user-facing application that owns the LLM and decides which servers to trust.

Client / Server

The two endpoints of one MCP session; client drives, server offers. 1:1 per session.

Capability

A declared feature, exchanged at `initialize`; unadvertised capabilities must not be used.

Primitive

A tool, resource, or prompt — the three things a server exposes.

Tool / Resource / Prompt

Model-controlled function / application-pulled data / user-invoked template.

Notification

An id-less JSON-RPC message; no response expected.

Reverse request

A server-originated request to the host: sampling, roots, or elicitation.

Transport

The byte channel under JSON-RPC: stdio or Streamable HTTP.

Handler

User code registered for a tool/resource/prompt; runs on a worker thread under the cooperative-cancellation contract.

Dual error channel

Protocol failures use JSON-RPC `error`; tool-run failures use an `isError:true result`.